INFORME TÉCNICO DEL PROYECTO Y SU EJECUCIÓN EN DOCKER

El desarrollo de este laboratoio se basa en completar las funciones de publicaciones y amistad del proyecto proporcionado por el JP Lewis. La aplicación permite listar, crear y editar posts, eliminar publicaciones con permisos de administrador y gestionar solicitudes entre cuentas activas como lo mostró en la pizarra. La seguridad se comprueba en el servidor, de modo que cambiar un identificador en una URL o en un formulario no concede acceso a información ajena.

El proyecto conserva Spring Boot 2.7.18 y la estructura Java del laboratorio. Se añadieron servicios para las reglas de negocio, formularios de entrada con validación, pruebas automatizadas y los archivos necesarios para construir la aplicación y MySQL mediante Docker Compose.

Tecnologías utilizadas

Según `pom.xml`, la aplicación usa:

Tecnología	Papel en el proyecto
Java 17 y Maven	Lenguaje y gestión de dependencias y compilación.
Spring Boot 2.7.18 y Spring MVC	Arranque de la aplicación, rutas HTTP y controladores.
Spring Data JPA	Repositorios y persistencia de las entidades. El proyecto también declara Spring Data JDBC.
MySQL	Base de datos configurada mediante el controlador JDBC de MySQL.
Spring Security	Autenticación, autorización por roles y codificación BCrypt de contraseñas.
Thymeleaf	Generación de páginas HTML a partir de plantillas y datos del modelo.
Spring Session JDBC	Configuración de sesiones almacenadas mediante JDBC.
Bootstrap, jQuery, Popper y Font Awesome	Recursos de interfaz incluidos en `static/`.
Spring Validation y Spring Boot Test	Dependencias para validación y pruebas. La prueba incluida solo intenta cargar el contexto de Spring.

Problemas encontrados en la base

La revisión inicial mostró diferencias entre las rutas de los controladores y los enlaces de las vistas. La creación de posts duplicaba el segmento de la URL, algunos redireccionamientos apuntaban a vistas inexistentes y el menú llamaba métodos de usuario que no existían. El borrado se ejecutaba con GET y no coincidía con la ruta protegida en la configuración de seguridad.

El guardado recibía una entidad completa sin verificar quién era su autor. Esto hacía necesario revisar tanto la pantalla de edición como la operación que guarda los cambios. También faltaban el formulario de acceso en su ruta prevista y el flujo de solicitudes de amistad.

Criterio de permisos

Se mantuvieron los roles admin y user de la base. El código original reservaba la eliminación al administrador. Como complemento, un usuario puede editar sus propios posts y el administrador puede editar cualquiera. Las solicitudes de amistad funcionan igual para ambos roles. Además, ser administrador no permite responder en nombre del destinatario.

Acción	USUARIO	ADMIN
Listar posts	Publicados y borradores propios	Todos
Crear posts	Sí	Sí
Editar posts	Solo propios	Cualquiera
Eliminar posts	No	Sí
Editar perfil	Solo propio	Solo propio
Listar cuentas	No	Sí
Responder solicitud	Solo destinatario	Solo destinatario

Visitante	Registros que consulta el controlador	Resultado previsto
`admin`	Todos	Publicaciones y borradores de todos los autores.
`user`	Todos	Publicaciones y borradores de todos los autores.
Anónimo	Todos; la ruta está permitida	La plantilla puede fallar al evaluar el rol de la sesión.

Funciones y mejoras del código

Publicaciones

PostController concentra las rutas de la interfaz y delega las decisiones en PostService. Al guardar un post nuevo, el servidor asigna como autor a la cuenta autenticada. Al editar uno existente, carga el registro, comprueba la autoría o el rol de administrador y modifica únicamente título, resumen, contenido y estado de publicación. El autor original se conserva incluso si la edición la realiza un administrador.

PostForm limita el título a 75 caracteres, el resumen a 255 y el contenido a 10000. Título y contenido son obligatorios. Los errores se muestran en el formulario. La eliminación usa POST y exige rol admin. Un identificador inexistente produce 404. Intentar editar un post ajeno produce 403, las vistas muestran el texto con escape de HTML.

Sesión y autenticación

El acceso utiliza el correo de la cuenta y una contraseña verificada con BCrypt. Spring Security consulta las tablas user y role y rechaza las cuentas inactivas. Después del ingreso redirige al listado de posts. La identidad y el rol usados por las reglas de negocio se recuperan a partir de la cuenta autenticada, sin depender de datos enviados por el navegador.

Las sesiones se almacenan con Spring Session JDBC y vencen tras 30 minutos de inactividad. La configuración protege la cookie con HttpOnly y SameSite Lax. El indicador Secure se puede activar cuando la aplicación se sirva por HTTPS. Al iniciar sesión se renueva el identificador; al salir se invalida la sesión y se eliminan las cookies de sesión.

Se conserva la protección CSRF de Spring Security. Los formularios Thymeleaf con th:action incorporan el token correspondiente. Guardar un post, enviar una solicitud o cerrar sesión sin ese token debe ser rechazado. Ocultar un botón sirve para orientar al usuario, pero la autorización se vuelve a comprobar al recibir la operación.

Inicio de sesión y contraseñas

En `Config/WebSecurityConfig.java`, Spring Security está configurado para:

1. Mostrar la página de acceso en `/loginForm`.
2. Procesar el formulario mediante `POST /processlogin`.
3. Buscar al usuario por el correo introducido en el campo `username`, consultando `email`, `password` y `active` de la tabla `user`.
4. Obtener sus autoridades mediante una consulta que une `user` con `role`.
5. Tras autenticarse, redirigir a `/redirectByRol`.

XML

```xml
<input type="hidden" name="_csrf" value="TOKEN_GENERADO">
```

Ejemplos concretos

El formulario de acceso en `templates/index.html` envía las credenciales a `/processlogin`:

```xml
<form th:action="@{/processlogin}" method="post">
    <!-- username y password -->
</form>
```

El formulario de posts en `templates/post/form.html` envía a `/post/guardar`:

```xml
<form method="post" th:action="@{'/post/guardar'}"
      th:object="${post}">
    <!-- campos del post -->
</form>
```

Solicitudes de amistad

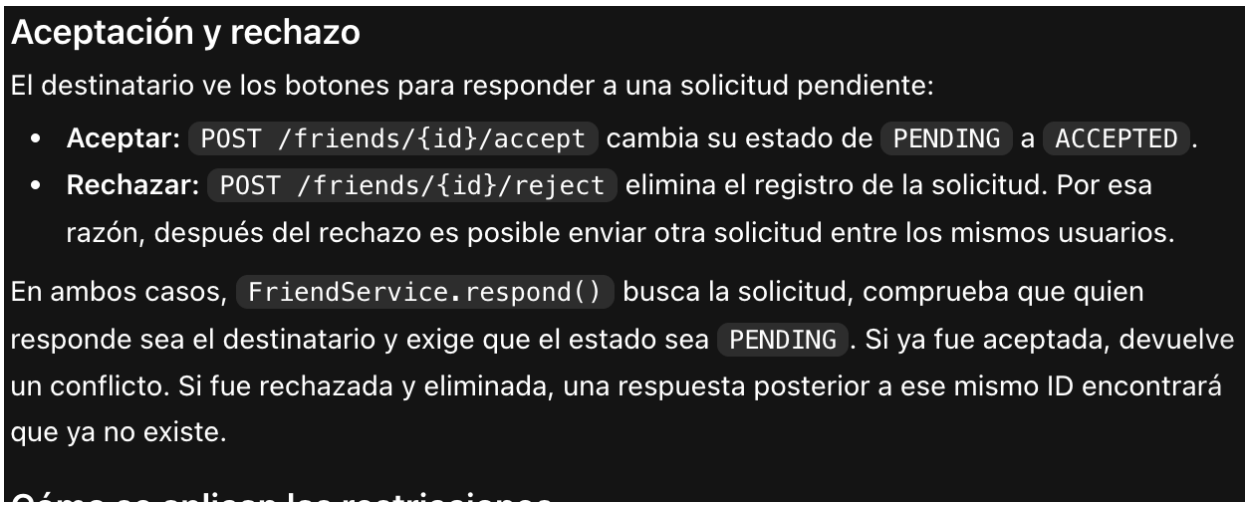
La nueva tabla friend_request guarda remitente, destinatario, fecha y estado. Una solicitud pendiente usa PENDING y una amistad aceptada usa ACCEPTED. La pareja se representa con los identificadores menor y mayor y tiene una restricción única. Así se impide guardar dos veces la relación aunque las solicitudes se envíen en sentidos opuestos.

El envío bloquea las dos cuentas en orden fijo dentro de una transacción. No admite solicitudes a uno mismo, a cuentas inexistentes o a cuentas inactivas. Para aceptar o rechazar se bloquea la solicitud y se verifica que la cuenta autenticada sea su destinatario. Al rechazar, la fila se elimina, esto permite solicitar amistad nuevamente.

Envío

1. El usuario abre `GET /friends`. La vista muestra usuarios activos y las solicitudes en las que participa.
2. Selecciona un destinatario y envía el formulario a `POST /friends/send`.
3. El controlador obtiene la identidad del emisor desde la autenticación y pasa su correo y el ID del destinatario a `FriendService.send()`.
4. El servicio comprueba que el emisor esté activo y que el destinatario exista y esté activo.
5. Si las validaciones pasan, guarda una solicitud con estado `PENDING` y redirige a `/friends`.

El emisor **no se toma de un campo enviado por el formulario**: se determina a partir de la sesión autenticada.



Perfil y organización

El formulario de perfil permite cambiar nombre, apellido, presentación y descripción. No admite asignar el rol, la contraseña, el estado o el identificador. Se retiraron las rutas incompletas de administración que guardaban entidades de usuario completas. No se añadió un módulo de altas y bajas, porque no era parte de las funciones solicitadas.

Contenerización y forma de uso

La entrega incluye Dockerfile, compose.yaml, .dockerignore, .env.example y un script de inicialización. El diseño utiliza dos servicios: app ejecuta Spring Boot y db ejecuta MySQL 8.4. Separarlos permite conservar la base aunque se reconstruya la aplicación.

Construcción de la imagen

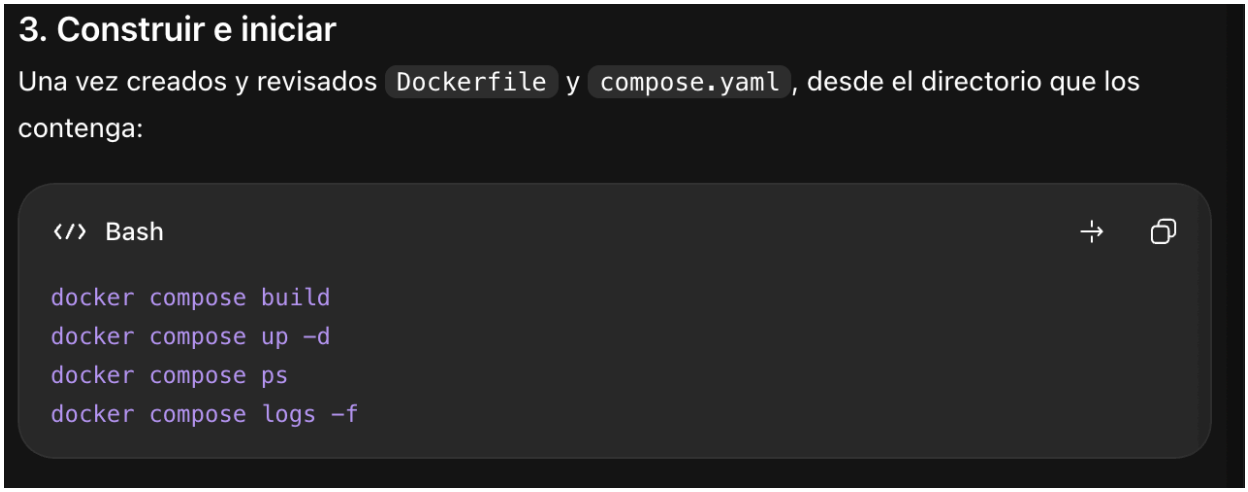
El Dockerfile tiene dos etapas. La primera utiliza Maven con Java 17, descarga dependencias y ejecuta mvn verify antes de empaquetar el JAR. La segunda incluye Java 17 JRE y el archivo generado. El proceso se ejecuta con un usuario sin privilegios. Los archivos temporales, el directorio target y el archivo .env se excluyen del contexto de construcción.

Compose monta los datos en un volumen persistente y ejecuta el script SQL solo al crear una base vacía. El script conserva los registros del laboratorio, corrige los comentarios SQL e incorpora la tabla de solicitudes. También asigna contraseñas conocidas a tres cuentas de demostración.

El servicio app espera a que MySQL supere una consulta de salud. La conexión usa el nombre db, que es resoluble dentro de la red de Compose. El puerto de MySQL no se publica y la aplicación se expone en 127.0.0.1:8080. Las claves de ejemplo son exclusivamente para el uso local del laboratorio.

Pasos para ejecutar

cp .env.example .env
docker compose up --build -d
docker compose logs -f app



Cuentas de demostración

Administrador: fermin.shanahan@example.com.

Usuarios: abauch@example.com y edison71@example.com.

Las tres cuentas utilizan la contraseña Lab8Demo2026! y sus hashes se generan con BCrypt. Las demás cuentas conservan los hashes del proyecto original.

Recorrido de comprobación

Ingresar con abauch@example.com, crear un borrador y editarlo. Luego enviar una solicitud al usuario edison71@example.com. Cerrar sesión, ingresar con la segunda cuenta y aceptar la solicitud. Ambas cuentas deben verla como amistad. Finalmente, ingresar como administrador y comprobar la edición de posts ajenos y la eliminación. La opción de eliminar no aparece para user y el servidor también bloquea el intento directo.

Alternativa de desarrollo

Fuera de Docker se requiere JDK 17, Maven y una base MySQL inicializada con docker/init/01-schema.sql. Las variables DB_URL, DB_USER y DB_PASSWORD configuran el acceso. Las pruebas de mvn clean verify usan H2 y no requieren una instalación de MySQL.

Límites del apoyo de IA

La IA resultó útil para organizar la revisión, proponer controles y preparar casos de prueba. Las decisiones finales deben apoyarse en el funcionamiento del proyecto. Los ejemplos anteriores no sustituyen la explicación de las reglas de acceso, de las relaciones de la base o de las variables de configuración. Tampoco se atribuyen al estudiante experiencias que no constan en el trabajo realizado.

Una recomendación requiere ajustes cuando se aplica al código existente. En esta entrega se conservaron las tecnologías de la base y se concentró el cambio en las funciones solicitadas. Se comprobó la coherencia entre controladores, servicios, formularios y plantillas en lugar de aceptar fragmentos aislados.

Verificación y resultados de la entrega

Pruebas automatizadas

La suite de integración utiliza Spring Boot, MockMvc y H2 en modo MySQL. Incluye 16 pruebas que ejercitan los controladores con los filtros de seguridad y la persistencia. Comprueba el listado según rol, la creación con autor autenticado, la edición propia, la edición administrativa, el bloqueo de cambios ajenos y el borrado exclusivo para admin.

También cubre validaciones y recursos inexistentes, el rechazo de operaciones sin CSRF, la protección del listado de cuentas y los intentos de cambiar el rol o modificar otra cuenta desde el perfil. Para amistad revisa autorreferencias, cuentas inactivas o inexistentes, duplicados recíprocos, aceptación exclusiva por el destinatario, rechazo y nuevo envío.

Resultado registrado

Se ejecutaron las 16 pruebas de integración con resultado satisfactorio: 0 fallos, 0 errores y 0 pruebas omitidas. La compilación se realizó en el entorno disponible con JDK 26 y destino Java 17, la imagen de Docker fija Java 17. La configuración de Compose pasó docker compose config --quiet. El detalle reproducible y el resumen de Surefire se adjuntan dentro del proyecto. También se inició el JAR en un servidor HTTP real con H2 y se verificaron acceso con tres cuentas, creación, escape de HTML, envío y aceptación de amistad, borrado según rol y cierre de sesión.

Alcance pendiente

No se pudo arrancar Docker en el equipo de preparación: el daemon no estaba disponible y la instalación local de Docker Desktop carecía del ejecutable necesario. Por ello, la entrega no afirma haber validado la construcción de la imagen, la inicialización con MySQL 8.4 ni la conservación del volumen en una ejecución real.

Las pruebas con H2 verifican comportamiento de aplicación, pero no reproducen todas las diferencias del motor MySQL ni sustituyen una prueba de concurrencia bajo carga. Los bloqueos y la clave única están implementados; la suite comprueba duplicados secuenciales y recíprocos, no una carrera de peticiones simultáneas.

LINK DE CONVERSACIÓN DE CODEX:

https://chatgpt.com/s/cx_6aed3e273c7481918a7678457f21854c